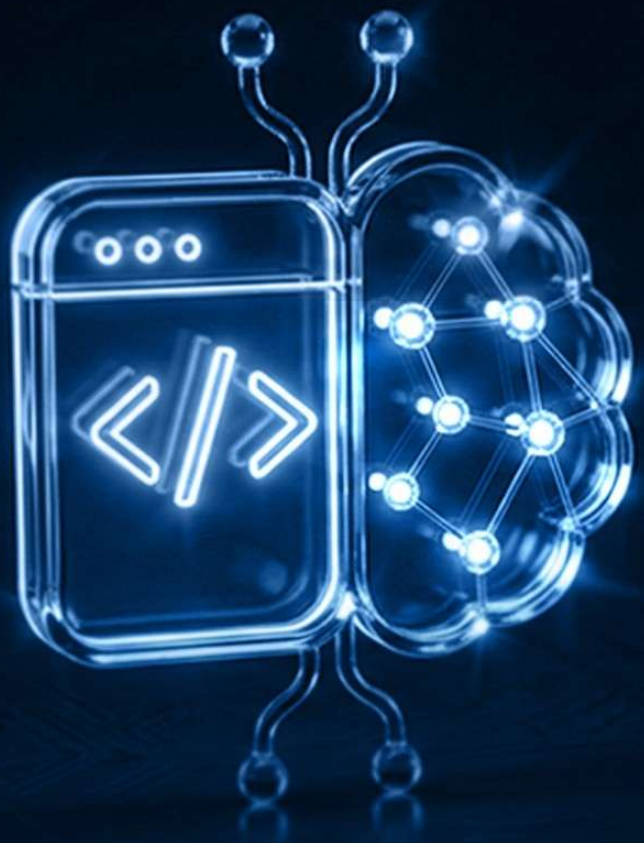


دوره آموزشی

GenAI

مهندسی نرم‌افزار در عصر هوش مصنوعی

مدرس: علیرضا ارومند



کارگاه شماره سه

معماری محیط توسعه و راه اندازی آن

از تولید کد تا مهندسی نرم‌افزار با AI



- تولید کد، فقط بخشی از چرخهٔ توسعه است
- هدف: توسعهٔ قابل‌فهم، قابل‌آزمون و قابل‌تکرار
- حرکت از فعالیت فردی به فرایند تیمی
- ترکیب Agent، ابزار، استاندارد و مسئولیت انسانی
- خروجی نهایی: یک روش توسعهٔ قابل‌اعتماد

مسئلهٔ واقعی توسعه با چند Agent

- چند توسعه‌دهنده با عامل‌های مستقل
- برداشت‌های متفاوت از معماری و نیازمندی
- تولید تغییرات ناسازگار در بخش‌های مختلف
- موفقیت ظاهری Agentها بدون موفقیت محصول
- نیاز به هماهنگی، شواهد و مالکیت انسانی



چرخهٔ کامل توسعهٔ نرم‌افزار با AI



- Product Discovery و تحلیل نیازمندی
- Specification و طراحی معماری
- برنامه‌ریزی، تقسیم کار و Implementation
- Verification و Product Validation
- Review، Release و Operations
- بازگشت شواهد به نیازمندی و طراحی

یک Workflow برای همهٔ کارها مناسب نیست

- Feature جدید: نیازمندی تا پذیرش
- Bug Fix: بازتولید، علت‌یابی و Regression
- تغییر معماری: گزینه‌ها، ADR و مهاجرت
- Spike: فرضیه، آزمایش و گزارش
- کنترل‌ها متناسب با پیچیدگی و ریسک تغییر



تفاوت Skill، Agent، Model و Tool



- Model: تحلیل و تولید پاسخ
- Agent Runtime: مدیریت اجرای فعالیت
- Skill: دانش رویه‌ای و روش انجام کار
- Tool / CLI / API: انجام عملیات واقعی
- Workflow: ترتیب و هماهنگی مراحل
- Human: مسئولیت تصمیم و پذیرش

تفاوت Tool، Process و Control

- Tool: اجرای یک عملیات مشخص
- Process: ترتیب، وابستگی و مسئولیت فعالیت‌ها
- Control: شرط اجازهٔ عبور از یک مرحله
- Spec Kit و Codex دارای مسئولیت‌های متفاوت
- CI و Git Policies برای کنترل مستقل
- وجود ابزار، معادل وجود فرایند مهندسی نیست



معماری محیط توسعهٔ BlogSpecLab



- VS Code و Codex برای فعالیت توسعه
- Spec Kit برای Specification و Tasks
- Git برای ثبت و جداسازی تغییرات
- PR و CI برای بازبینی و کنترل کیفیت
- Release و Monitoring برای رفتار عملیاتی
- مستندات نسخه‌دار به‌عنوان منابع مشترک

گزارش Agent چرا برای پذیرش کافی نیست؟

- امکان موفقیت تست‌های ناقص یا اشتباه
- بازبینی کیفیت و سازگاری معماری
- کنترل امنیت و آسیب‌پذیری‌ها
- بررسی Regression و اثر تغییر بر قابلیت‌های دیگر
- سنجش رفتار واقعی در برابر Acceptance Criteria
- استقلال شواهد از ادعای Agent



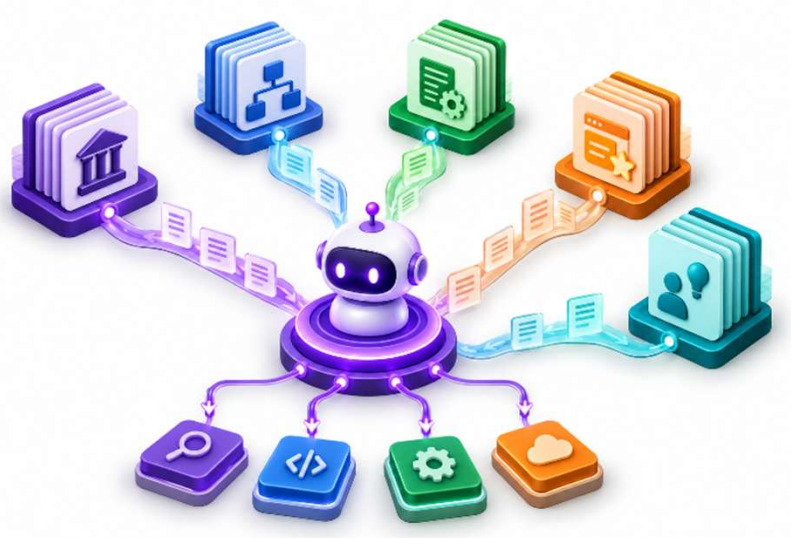
Merge Gate، Release Gate و DORA



- Merge Gate: پذیرش تغییر در شاخهٔ اصلی
- Release Gate: عرضهٔ ایمن به مصرف‌کننده
- CI، Code Review و Policy پیش از Merge
- پذیرش محصول، Rollback و Monitoring پیش از Release
- DORA: سنجش سرعت و پایداری تحویل
- معیارهای جریان تحلیل تا تست، مکمل DORA

معماری Context و منابع حقیقت

- اطلاعات پروژه در منابع مستقل و معتبر
- تفکیک قواعد پایدار، تصمیم‌ها و نیازمندی‌ها
- جلوگیری از تکرار و تعارض مستندات
- Context مرتبط با فعالیت جاری
- منابع نسخه‌دار و قابل‌ردیابی
- تفاوت وجود اطلاعات با استفادهٔ صحیح Agent



مرز تصمیم محصولی و تصمیم فنی



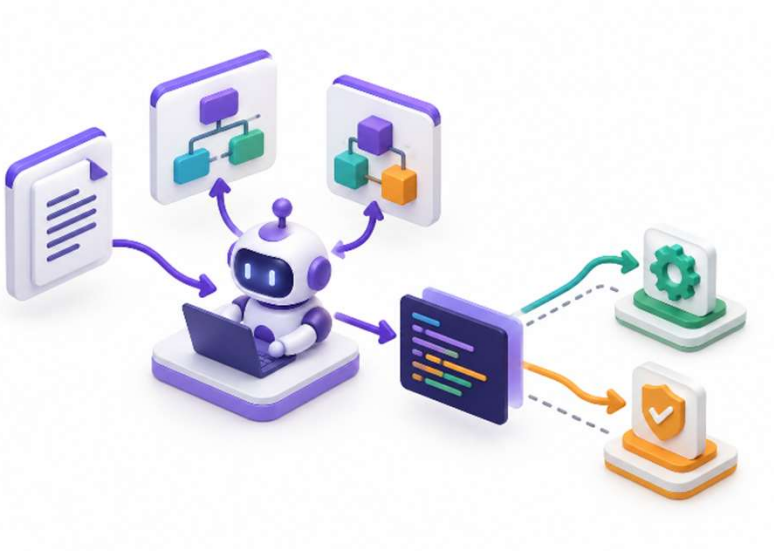
- Product Specification: رفتار مورد انتظار
- Constitution: اصول فراگیر و پایدار
- ADR: انتخاب فنی و دلایل تصمیم
- AGENTS.md: روش کار Agent در پروژه
- Tests: شواهد رعایت رفتار و قرارداد
- پرهیز از انتقال Business Rule به فایل نامناسب

Constitution و ADR؛ دو سطح تصمیم

- Constitution: اصول بنیادی مهندسی
- ADR: تصمیم فنی در یک زمینهٔ مشخص
- ثبت گزینه‌های جایگزین و Trade-offها
- مشخص بودن وضعیت و اعتبار تصمیم‌ها
- مدیریت تعارض و تغییر تصمیم‌های قبلی
- Design Review در کنار آزمون‌های خودکار



AGENTS.md و کنترل رفتار Codex



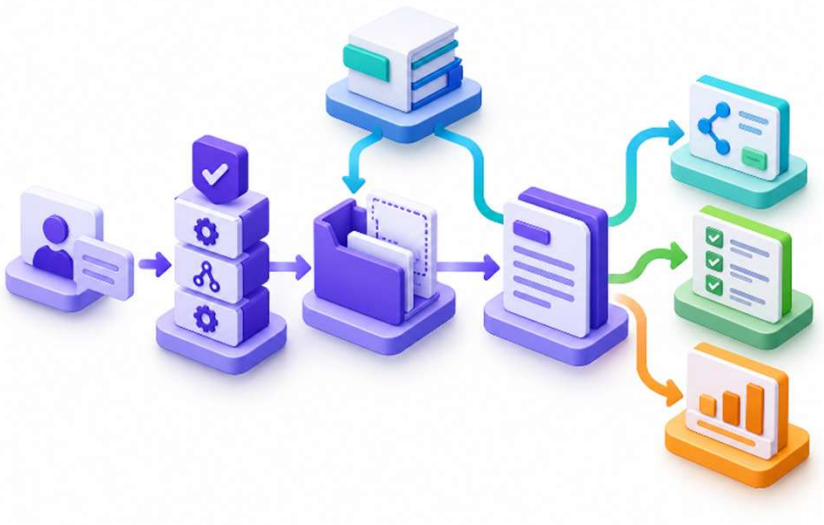
- راهنمای پایدار همکاری Agent با پروژه
- معرفی ساختار، منابع و قراردادهای معتبر
- قواعد Build، Test و نحوهٔ گزارش نتیجه
- محدودهٔ تغییرات و موارد نیازمند تأیید
- تفاوت Instruction با Enforced Policy
- کنترل واقعی از طریق ابزار و CI

Agent Skills؛ دانش رویه‌ای قابل‌استفادهٔ مجدد

- استاندارد باز Agent Skills
- ساختار اصلی SKILL.md و Frontmatter
- Name و Description برای کشف Skill
- فراخوانی صریح یا انتخاب خودکار
- References، Scripts و Assets اختیاری
- تفاوت Skill با AGENTS.md و Tool



کالبدشکافی Skill واقعی speckit-specify



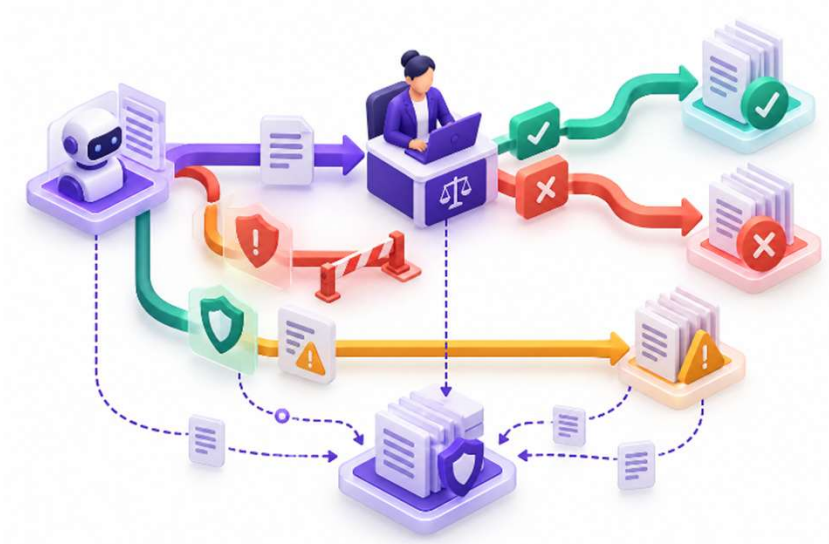
- دریافت ورودی و بررسی Pre-Execution
- بررسی Hookهای پیش از اجرا
- ایجاد مسیر Feature و انتخاب Template
- خواندن Constitution و تولید spec.md
- تولید Checklist و بازبینی کیفیت
- Post-Hook، گزارش نهایی و Done When

ترکیب Skill با CLI اختصاصی

- Skill: تعریف روش انجام فعالیت
- CLI: تولید قطعی فایل‌های استاندارد
- Codex: آماده‌سازی ورودی و فراخوانی CLI
- Dry-run، اعتبارسنجی و جلوگیری از بازنویسی
- کنترل نتیجه با Build، Test و Git Diff
- ممنوعیت جایگزینی دستی بدون مجوز



مرز اعتماد، Human Gate و مدیریت شکست



- تفاوت Draft با خروجی مورد تأیید
- کنترل امنیتی اجباری و اصل Fail Closed
- ادامهٔ محدود با هشدار برای عملیات کم‌خطر
- توقف در تغییرات حساس یا ابهام امنیتی
- ثبت خطا، شواهد و تصمیم انسانی
- آماده‌سازی ورود به Multi-Agent و Workflow

Multi-Agent چیست و چرا به آن نیاز داریم؟

- تفاوت Multi-Role، Single-Agent و Multi-Agent
- عامل‌های مستقل با Context و مسئولیت مشخص
- تفکیک وظایف و اجرای موازی
- مزایای تمرکز و استقلال بررسی
- هزینه‌های هماهنگی، امنیت و مصرف منابع



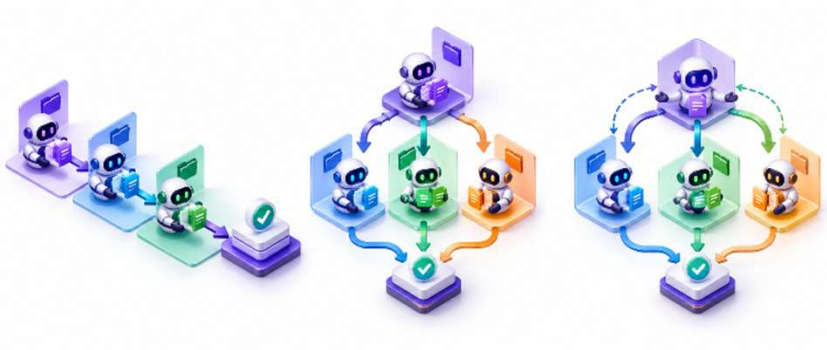
معماری همکاری Agentها در توسعهٔ نرم‌افزار



- Coordinator و عامل‌های تخصصی
- Task Contract و محدودهٔ اختیار
- Context و منابع مشترک معتبر
- Handoff، خروجی و معیار پایان
- Integration، CI و Human Review

الگوهای Multi-Agent و ارتباط با Workflow

- Sequential Handoff
- Parallel Delegation
- Coordinator / Specialists
- تداخل تغییرات و مالکیت فایل‌ها
- تفاوت Multi-Agent با Workflow
- انتخاب الگو بر اساس وابستگی و ریسک



چه زمانی توسعهٔ Multi-Agent رو موازی کنیم؟



- استقلال واقعی وظایف، پیش‌شرط Parallelism
- قراردادهای مشترک و تأییدشده قبل از اجرا
- تفاوت Context Isolation و Workspace Isolation
- Branch و Worktree برای جداسازی تغییرات
- مدیریت تغییر قرارداد و تعارض هنگام Integration
- انتخاب Sequential یا Parallel بر اساس هزینه و ریسک

هماهنگی Agentها و تحویل کنترل‌شدهٔ کار

- نقش Coordinator در تقسیم و پیگیری وظایف
- تعریف Task Contract و محدودهٔ اختیارات
- اجرای ترتیبی Development و Test Agent
- Handoff همراه با خروجی و شواهد واقعی
- مدیریت شکست، اصلاح و اجرای مجدد
- Human Approval در تصمیم‌های حساس



پیکربندی Agentهای تخصصی در Codex



- تعریف Agentهای اختصاصی در codex/agents.
- تنظیم Name، Description و Developer Instructions
- تفکیک نقش Developer و Test Reviewer
- تعیین Context، ابزارها و محدودهٔ مسئولیت
- تفاوت دستورالعمل Agent با محدودیت اجرایی
- کنترل خروجی و گزارش‌دهی به Coordinator

Workflow چیست و چه مسئله‌ای را حل می‌کند؟

- تبدیل مراحل پراکنده به فرایند قابل‌تکرار
- تعریف ترتیب اجرا، وابستگی‌ها و شرایط عبور
- تفاوت Workflow با Skill، Hook و Multi-Agent
- کنترل جریان با شرط، تکرار و مدیریت خطا
- Human Gate و توقف برای تصمیم انسانی
- ثبت وضعیت و امکان ادامهٔ اجرای متوقف‌شده



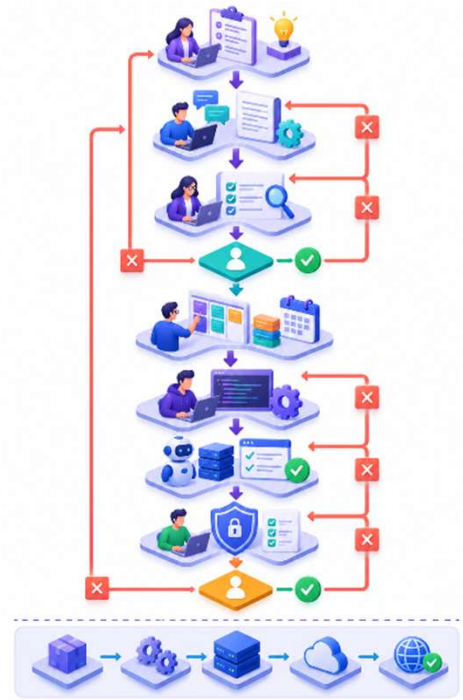
ساختار و اجرای Workflow در Spec Kit



- تعریف Workflow در YAML و تعیین ورودی‌ها
- Stepهای shell، prompt، command و gate
- Workflow آمادهٔ speckit و شش مرحلهٔ اصلی
- فرمان‌های status، info، run و resume
- تفاوت وضعیت‌های Paused، Completed و Aborted
- اجرای واقعی Human Gate و ثبت Run ID

توسعهٔ Workflow برای چرخهٔ کامل SDD

- افزودن Clarify و رفع ابهام پس از Specify
- افزودن Analyze برای بررسی سازگاری Artifactها
- Human Gate برای تأیید Specification و Plan
- اجرای ترتیبی Agent توسعه و Agent بازبین
- Integration Test و بازبینی امنیتی پس از Implement
- Final Approval و تفکیک Workflow از CI/CD



معماری کلان محیط عملیاتی توسعه با AI در سازمان

آنچه در کلاس ساختیم، هستهٔ این معماری است؛ بخش‌های دیگر باید در مقیاس سازمانی طراحی و پیاده‌سازی شوند.

